Symbol	Type	Meaning
$[N]$	$\subset \mathbb{N}$	The set $\{1, \ldots, N\}$ for $N \in \mathbb{N}$.
$\Sigma, \underline{\Sigma}, \overline{\Sigma}$	alphabet	Σ is a set of symbols, $\underline{\Sigma} \stackrel{\text{def}}{=} \Sigma \cup \{\text{BOS}\}$, $\overline{\Sigma} \stackrel{\text{def}}{=} \Sigma \cup \{\text{EOS}\}$
$y, \underline{y}, \overline{y}$	$\in \Sigma$	A symbol, element of Σ, $\underline{\Sigma}$, or $\overline{\Sigma}$.
$\boldsymbol{y}$	$\in \Sigma^*$	A string over Σ.
$\boldsymbol{y}_j^i$	$\in \Sigma^*$	A substring of $\boldsymbol{y}$, a string.
$[\![y]\!]$	$\in \{0,1\}^{\lvert\Sigma\rvert}$	One-hot encoding of the symbol $y \in \Sigma$.
D	$\in \mathbb{N}$	Size of the contextual representations in the transformer or RNN.
$\boldsymbol{\Delta}^{N-1}$	$\subseteq \mathbb{R}^N$	The $N-1$-dimensional probability simplex.
f	$\mathbb{R}^D \times \mathbb{R}^D \to \mathbb{R}$	A scoring function.
$\mathbf{f}$	$\mathbb{R}^N \to \boldsymbol{\Delta}^{N-1}$	A normalization function.
Q, K, V, O	$\mathbb{R}^D \to \mathbb{R}^D$	The query, key, value, and output functions.
F	$\mathbb{R}^D \to \mathbb{R}^D$	The final transformer LM transformation function.
enc	$\Sigma^* \to \mathbb{R}^D$	The string representation function.
$\mathbf{r}_\Pi$	$\underline{\Sigma} \times \mathbb{N} \to \mathbb{R}^D$	The position-augmented representation function.
L	$\in \mathbb{N}$	Number of layers.
H	$\in \mathbb{N}$	Number of heads.
$\mathcal{H}$	$\mathbb{R}^{HD} \to \mathbb{R}^D$	The head combining function.
$(\cdot\,; \cdots ;\cdot)$		Vertical concatenation operator of vectors or matrices.

Table 1: A summary of the notation used in the paper.

As before, we define a **run** in the 2PDA as a sequence of consecutive transitions. A run is called **accepting** if it ends in a final configuration. The yield of an accepting run is the sequence of symbols $y \in \Sigma$ that the 2PDA scans (or outputs) during the run. And the **weight** of an accepting run, $\mathrm{w}(\boldsymbol{\pi})$, is the product of the weight of its transitions,

$$\mathrm{w}(\boldsymbol{\pi}) \stackrel{\text{def}}{=} \prod_{n=1}^{N} w_n, \tag{20}$$

where w_n is the weight of transition τ_n. Finally, the **stringsum** of string $\boldsymbol{y}$ is defined as the sum of the weights of all accepting runs that yield $\boldsymbol{y}$, i.e., $\Pi(\mathcal{P}, \boldsymbol{y})$:

$$\mathcal{P}(\boldsymbol{y}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{\pi} \in \Pi(\mathcal{P}, \boldsymbol{y})} \mathrm{w}(\boldsymbol{\pi}). \tag{21}$$

C.4 Transformer Language Models

Transformer LMs are LMs whose conditional distributions $p(\overline{y}_t \mid \boldsymbol{y}_{<t})$ are computed by a *transformer*. A transformer is a composition of multiple transformer *layers*, each of which implements the *attention mechanism*. We give definitions of these building blocks in what follows. Our formalization and notation closely follows Svete and Cotterell (2024).

Notation. We use bold unitalicized letters such as $\mathbf{x} \in \mathbb{R}^D$ to denote real-valued vectors and italicized letters $x_j \in \mathbb{R}$ for their entries. Capital bold letters such as $\mathbf{X} \in \mathbb{R}^{N \times D}$ denote matrices. All vectors are *column* vectors unless transposed. We define the vertical stacking operator $(\cdot\,; \cdots ;\,\cdot)$, which denotes the vertical concatenation of the D-dimensional *column* vectors $\mathbf{x}_1, \ldots, \mathbf{x}_N$ into a ND-dimensional vector $(\mathbf{x}_1; \cdots ; \mathbf{x}_N) \in \mathbb{R}^{ND}$ and the concatenation of the D-dimensional *row* vectors $\mathbf{x}_1^\top, \ldots, \mathbf{x}_N^\top$ into a matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ with N rows and D columns. Given the matrix $\mathbf{X} = \left(\mathbf{x}_1^\top; \cdots ; \mathbf{x}_N^\top\right)$, we write $\mathbf{X}_n = \left(\mathbf{x}_1^\top; \cdots ; \mathbf{x}_n^\top\right)$ for the submatrix composed of the first n rows. We call a function $f : \mathbb{R}^D \times \mathbb{R}^D \to \mathbb{R}$ whose purpose is to evaluate the compatibility of two vectors a **scoring function**. A **normalization function** $\mathbf{f} \colon \mathbb{R}^N \to \boldsymbol{\Delta}^{N-1}$ maps vectors in $\mathbb{R}^N$ to N probabilities. Here, $\boldsymbol{\Delta}^{N-1} \stackrel{\text{def}}{=} \left\{\mathbf{x} \in [0,1]^N \mid \sum_{n=1}^N x_n = 1\right\}$ is the $N-1$-dimensional probability simplex. This notation is summarized in Tab. 1.

The Attention Mechanism. The attention mechanism works as follows. It takes a **query** vector $\mathbf{q} \in \mathbb{R}^D$ and two matrices: The matrix $\mathbf{K} \in \mathbb{R}^{N \times D}$ of **keys** and the matrix $\mathbf{V} \in \mathbb{R}^{N \times D}$ of **values** and computes a weighted average of the value vectors based on the compatibilities of the key vectors to the query vector, as scored by a scoring function f. A formal definition is given below.

Definition C.4 (Attention Mechanism)**.** *The **attention mechanism*** $\texttt{Att}\colon \mathbb{R}^D \times \mathbb{R}^{N\times D} \times \mathbb{R}^{N\times D} \to \mathbb{R}^D$ *is defined as*

$$\texttt{Att}\left(\mathbf{q}, \mathbf{K}, \mathbf{V}\right) \stackrel{\text{def}}{=} \sum_{n=1}^{N} s_n \mathbf{v}_n \tag{22}$$

where $\mathbf{q} \in \mathbb{R}^D$ *be a query vector and let* $\mathbf{K} = \left(\mathbf{k}_1^\top; \cdots; \mathbf{k}_N^\top\right) \in \mathbb{R}^{N\times D}$ *and* $\mathbf{V} = \left(\mathbf{v}_1^\top; \cdots; \mathbf{v}_N^\top\right) \in \mathbb{R}^{N\times D}$ *be matrices of keys and values, respectively, and*

$$\mathbf{s} \stackrel{\text{def}}{=} \mathbf{f}\left(f\left(\mathbf{q}, \mathbf{k}_1\right), \ldots, f\left(\mathbf{q}, \mathbf{k}_N\right)\right) \tag{23}$$

is the vector of normalized scores between the query $\mathbf{q}$ *and the keys in* $\mathbf{K}$*,* f *is a scoring function and* $\mathbf{f}$ *is a normalization function.*

Attention types. Attention weights are computed by normalizing the scores $f\left(\mathbf{q}, \mathbf{k}_1\right), \ldots, f\left(\mathbf{q}, \mathbf{k}_t\right)$. The choice of the projection function $\mathbf{f}$ determines the type of attention and has concrete implications on representational capacity (Hao et al., 2022). We focus on the **hard attention** projection function.

Definition C.5. ***Hard attention*** *is computed with the* $\mathrm{hardmax}$ *projection function:*

$$\mathrm{hardmax}\left(\mathbf{x}\right)_d \stackrel{\text{def}}{=} \begin{cases} \frac{1}{m} & \textbf{\textit{if}}\ d \in \mathrm{argmax}\left(\mathbf{x}\right) \\ 0 & \textbf{\textit{otherwise}} \end{cases} \tag{24}$$

for $d \in [D]$*, where* $\mathbf{x} \in \mathbb{R}^D$ *and* $m \stackrel{\text{def}}{=} |\,\mathrm{argmax}\left(\mathbf{x}\right)|$ *is the cardinality of the argmax set.*

Definition C.6. *A **multi-layer perceptron** (MLP)* $F\colon \mathbb{R}^D \to \mathbb{R}^D$ *is a function defined as the composition of elementary functions* $\mathbf{f}_1, \ldots, \mathbf{f}_L$

$$F\left(\mathbf{x}\right) \stackrel{\text{def}}{=} \mathbf{f}_L \circ \mathbf{f}_{L-1} \circ \cdots \circ \mathbf{f}_1\left(\mathbf{x}\right), \tag{25}$$

where each function $\mathbf{f}_\ell$ *for* $\ell \in [L]$ *is defined as*

$$\mathbf{f}_\ell(\mathbf{x}) \stackrel{\text{def}}{=} \boldsymbol{\beta}\left(\mathbf{W}_\ell \mathbf{x} + \mathbf{b}_\ell\right) \quad \ell \in [L-1] \tag{26a}$$

$$\mathbf{f}_L(\mathbf{x}) \stackrel{\text{def}}{=} \mathbf{W}_L \mathbf{x} + \mathbf{b}_L, \tag{26b}$$

where $\mathbf{W}_\ell \in \mathbb{R}^{D\times D}$ *is a square weight matrix specific to layer* ℓ*,* $\mathbf{b}_\ell \in \mathbb{R}^D$ *is a bias vector, and* $\boldsymbol{\beta}$ *is an element-wise non-linear activation function. The function* $\mathbf{f}_1$ *is called the **input layer**, the function* $\mathbf{f}_L$ *is called the **output layer**, and the function* $\mathbf{f}_\ell$ *for* $\ell = 2, \ldots, L-1$ *are called **hidden layers**.*[23]

The Transformer Architecture. A transformer layer uses the attention mechanism to compute augmented representations $\mathbf{z}_t = \texttt{Att}\left(\mathbf{q}_t, \mathbf{K}_t, \mathbf{V}_t\right)$ of the input representations $\mathbf{X}_t = \left(\mathbf{x}_1; \cdots; \mathbf{x}_t\right)$. The query $\mathbf{q}_t$, the keys $\mathbf{K}_t$, and values $\mathbf{V}_t$ are all transformations of the input representations $\mathbf{X}_t$.

Definition C.7. *Given query, key, value, and **output** functions* $Q, K, V, O\colon \mathbb{R}^D \to \mathbb{R}^D$*, a **transformer layer** is a function* $\mathcal{L}\colon \mathbb{R}^{T\times D} \to \mathbb{R}^{T\times D}$ *that computes*

$$\mathcal{L}\left(\mathbf{x}_1^\top; \cdots; \mathbf{x}_T^\top\right) = \left(\mathbf{z}_1^\top; \cdots; \mathbf{z}_T^\top\right) \in \mathbb{R}^{T\times D} \tag{27}$$

for $t \in [T]$ *where*

$$\mathbf{a}_t \stackrel{\text{def}}{=} \texttt{Att}\left(\mathbf{q}_t, \mathbf{K}_t, \mathbf{V}_t\right) + \mathbf{x}_t \in \mathbb{R}^D \tag{28a}$$

$$\mathbf{z}_t \stackrel{\text{def}}{=} O\left(\mathbf{a}_t\right) + \mathbf{a}_t \qquad \in \mathbb{R}^D. \tag{28b}$$

[23] Note that we refer to MLPs by the number of hidden layers, e.g., a one-layer-MLP is an MLP with one *hidden* layer.

Here, we define

$$\mathbf{q}_t \stackrel{\text{def}}{=} Q\left(\mathbf{x}_t\right) \qquad \in \mathbb{R}^D \tag{29a}$$

$$\mathbf{K}_t \stackrel{\text{def}}{=} \left(K\left(\mathbf{x}_1\right)^\top; \cdots; K\left(\mathbf{x}_t\right)^\top\right) \in \mathbb{R}^{t \times D} \tag{29b}$$

$$\mathbf{V}_t \stackrel{\text{def}}{=} \left(V\left(\mathbf{x}_1\right)^\top; \cdots; K\left(\mathbf{x}_t\right)^\top\right) \in \mathbb{R}^{t \times D}. \tag{29c}$$

Note: *For simplicity, we do not include layer normalization.*

The functions Q, K, V are usually implemented as linear transformations and O as an MLP. Some theoretical literature, however, also considers more general function classes, e.g., all smooth functions (Hahn, 2020).

Without further modification, the transformations applied by the transformer layer are position-invariant, which necessitates the addition of explicit positional information.

Definition C.8. *A symbol **representation function** is a function $\mathbf{r}\colon \underline{\Sigma} \to \mathbb{R}^{D_\mathbf{r}}$ and a **positional encoding** is a function $\Pi\colon \mathbb{N} \to \mathbb{R}^{D_\Pi}$. A **position-augmented** representation function $\mathbf{r}_\Pi\colon \underline{\Sigma} \times \mathbb{N} \to \mathbb{R}^D$ (with $D = D_\mathbf{r} + D_\Pi$) is defined as*

$$\mathbf{r}_\Pi\left(\underline{y}_t, t\right) \stackrel{\text{def}}{=} \left(\mathbf{r}\left(\underline{y}_t\right); \Pi\left(t\right)\right). \tag{30}$$

Definition C.9. *A **static encoding** $\mathcal{R}$ is a function $\mathcal{R}\colon \Sigma^T \to \mathbb{R}^{T \times D}$ defined for any $T \in \mathbb{N}$ as*

$$\mathcal{R}\left(\boldsymbol{y}\right) \stackrel{\text{def}}{=} \left(\mathbf{r}_\Pi\left(y_1, 1\right)^\top; \cdots; \mathbf{r}_\Pi\left(y_T, T\right)^\top\right). \tag{31}$$

Multiple transformer layers are stacked into a transformer, which computes the (deep) contextual representations of all symbols in the string.

Definition C.10. *For $L \in \mathbb{N}$, an L-layer **transformer** $\mathcal{T}$ is defined as*

$$\mathcal{T}\left(\mathcal{R}\right) \stackrel{\text{def}}{=} \mathcal{L}_L \circ \cdots \circ \mathcal{L}_1 \circ \mathcal{R}, \tag{32}$$

where $\mathcal{L}_\ell$ for $\ell \in [L]$ are transformer layers and $\mathcal{R}$ is a static encoding.

A transformer computes the contextual representations of the symbols $\boldsymbol{y} = y_1 \cdots y_T$ as

$$\left(\mathbf{z}_1^\top; \cdots; \mathbf{z}_T^\top\right) \stackrel{\text{def}}{=} \left(\mathbf{x}_1^{L\top}; \cdots; \mathbf{x}_T^{L\top}\right) \stackrel{\text{def}}{=} \mathcal{T}\left(\mathcal{R}\right)\left(\boldsymbol{y}\right). \tag{33}$$

If $\mathcal{R}$ is clear from the context or arbitrary, we will omit it as an argument to $\mathcal{T}$ and just write $\mathcal{T}\left(\boldsymbol{y}\right)$.

Definition C.11. *Given a transformer $\mathcal{T}$, a final representation transformation function $F\colon \mathbb{R}^D \to \mathbb{R}^D$, and a string $\boldsymbol{y} \in \Sigma^*$ with $|\boldsymbol{y}| = T$, we define the **encoding function** enc as*

$$\mathsf{enc}\left(\boldsymbol{y}\right) \stackrel{\text{def}}{=} F\left(\mathbf{z}_T\right) \tag{34}$$

where $\mathbf{z}_T$ is the representation of the Tth symbol in $\boldsymbol{y}$ computed by $\mathcal{T}$, i.e., $\left(\mathbf{z}_1^\top; \cdots; \mathbf{z}_T^\top\right) = \mathcal{T}\left(\boldsymbol{y}\right)$.

Transformer Language Models. So far, we have only defined how the transformer architecture can be used to compute the contextual representations of the symbols. To complete the definition, we define a transformer *language model* as follows.

Definition C.12. *A **transformer LM** $p_\mathcal{T}$ is the representation-based autoregressive LM with the representation function enc from Eq. (34). That is, $p_\mathcal{T}$ defines the conditional probability distributions*

$$p_\mathcal{T}\left(y_t \mid \boldsymbol{y}_{<t}\right) \stackrel{\text{def}}{=} \mathrm{sparsemax}(\mathbf{E}\,\mathsf{enc}\left(\boldsymbol{y}_{<t}\right))_{y_t}. \tag{35}$$